Two Pointer Technique Study Guide

1) Introduction to Two Pointer

The Two Pointer technique is a fundamental algorithmic approach that uses two pointers (indices) to traverse an array or list. These pointers typically start at different positions and move according to specific conditions to solve problems efficiently.

Key Characteristics:

- **Time Complexity**: Usually O(n) instead of $O(n^2)$
- **Space Complexity**: O(1) - constant extra space
- **Movement**: Pointers can move in same or opposite directions

Types of Two Pointer Approaches:

Opposite Direction (Convergent)

- One pointer starts at the beginning (left = 0)
- Other pointer starts at the end (right = array.length - 1)
- Pointers move towards each other

```java
int left = 0;
int right = arr.length - 1;

while (left < right) {
    // Process current elements
    if (condition) {
        left++;
    } else {
        right--;
    }
}
```

Same Direction (Sliding Window)

- Both pointers start at the beginning
- They move in the same direction with different speeds

```java
int slow = 0;
int fast = 0;

while (fast < arr.length) {
    // Process elements
    if (condition) {
        slow++;
    }
    fast++;
}
```

2) Use Cases of Two Pointer

2.1 Sorted Array Problems

Example: Two Sum in Sorted Array

```java
public int[] twoSum(int[] numbers, int target) {
    int left = 0;
    int right = numbers.length - 1;

    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) {
            return new int[]{left + 1, right + 1};
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{};
}
```

2.2 Palindrome Checking

Example: Valid Palindrome

```java
public boolean isPalindrome(String s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        while (left < right && !Character.isAlphaNumeric(s.charAt(left))) {
            left++;
        }
        while (left < right && !Character.isAlphaNumeric(s.charAt(right))) {
            right--;
        }

        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}
```

2.3 Array Manipulation

Example: Remove Duplicates

```java
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;

    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

2.4 Subarray Problems

Example: Container With Most Water

```java
public int maxArea(int[] height) {
    int left = 0;
    int right = height.length - 1;
    int maxWater = 0;

    while (left < right) {
        int width = right - left;
        int currentWater = Math.min(height[left], height[right]) * width;
        maxWater = Math.max(maxWater, currentWater);

        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }
    return maxWater;
}
```

2.5 Linked List Problems

Example: Detect Cycle (Floyd's Algorithm)

```java
public boolean hasCycle(ListNode head) {
    if (head == null || head.next == null) return false;

    ListNode slow = head;
    ListNode fast = head.next;

    while (slow != fast) {
        if (fast == null || fast.next == null) {
            return false;
        }
        slow = slow.next;
        fast = fast.next.next;
    }
    return true;
}
```

3) Practice Questions on Two Pointer

3.1 Basic Implementation

Example: Reverse Array

```java
public void reverseArray(int[] arr) {
    int left = 0;
    int right = arr.length - 1;

    while (left < right) {
        // Swap elements
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        left++;
        right--;
    }
}
```

3.2 String Manipulation

Example: Reverse Words in String

```java
public String reverseWords(String s) {
    char[] chars = s.toCharArray();
    int left = 0;

    for (int right = 0; right <= chars.length; right++) {
        if (right == chars.length || chars[right] == ' ') {
            reverse(chars, left, right - 1);
            left = right + 1;
        }
    }
    return new String(chars);
}

private void reverse(char[] chars, int left, int right) {
    while (left < right) {
        char temp = chars[left];
        chars[left] = chars[right];
        chars[right] = temp;
        left++;
        right--;
    }
}
```

3.3 Advanced Applications

Example: 3Sum Problem

```java
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1;
        int right = nums.length - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}
```

Quiz Questions

Question 1: What is the primary advantage of using the Two Pointer technique? a) It uses less memory b) It reduces time complexity from $O(n^2)$ to $O(n)$ c) It makes code easier to read d) It works only on sorted arrays

Answer: b) It reduces time complexity from $O(n^2)$ to $O(n)$

Question 2: In the convergent two-pointer approach, when do the pointers stop moving? a) When left pointer reaches the middle b) When right pointer reaches the beginning c) When left >= right d) When both pointers are equal

Answer: c) When left >= right

Question 3: Which of the following problems is NOT typically solved using two pointers? a) Finding pairs with given sum in sorted array b) Palindrome checking c) Binary search d) Removing duplicates from sorted array

Answer: c) Binary search

Question 4: In Floyd's cycle detection algorithm, what is the speed relationship between the two pointers? a) Both move at same speed b) Fast pointer moves twice as fast as slow pointer c) Slow pointer moves twice as fast as fast pointer d) Fast pointer moves three times as fast

Answer: b) Fast pointer moves twice as fast as slow pointer

Question 5: What is the space complexity of most two-pointer algorithms? a) O(n) b) O(log n) c) $O(n^2)$ d) O(1)

Answer: d) O(1)

Practice Coding Questions

Question 1: Write a Java method to find if there exists a pair of numbers in a sorted array that sum up to a given target value. Return true if such a pair exists, false otherwise.

Question 2: Implement a method to move all zeros in an array to the end while maintaining the relative order of non-zero elements. Use the two-pointer technique.

Question 3: Given a string, write a method to determine if it's a valid palindrome, considering only alphanumeric characters and ignoring case.

Question 4: Write a Java method to find the maximum number of consecutive 1s in a binary array after flipping at most one 0 to 1.

Question 5: Implement a method to find three numbers in an array that sum up to zero. Return all unique triplets (avoid duplicates in the result).